

9 Temporal Networks

The social network you had when you were 3 years old was probably significantly different to the one you have now (unless you're reading this at a very precocious age). Networks change over time, with nodes and links disappearing and reappearing. In this chapter we study the time variation of networks and in particular the resilience of macroscale properties (connectedness, paths) and mesoscale properties (communities) over time.

Reading

[1] **Chapter 8**, [2] **Chapter 16**. See [3] for temporal networks.

9.1 Percolation

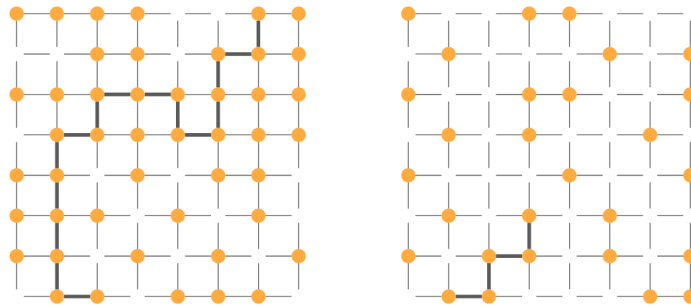


Figure 1: Removing a small versus a large fraction of the nodes.

Figure 1 shows the percolation model on a lattice. We imagine adding beads or opening valves to connect sites on the lattice. The question is, if sites are added with probability p what is the expected size of the largest cluster as a function of p ? Or a similar question, can a liquid flow from one side to the other along the links? It turns out (similarly to the ER model) there is a very sudden transition, a **phase transition**, between the case where there are lots of small disconnected clusters and one **giant cluster**.

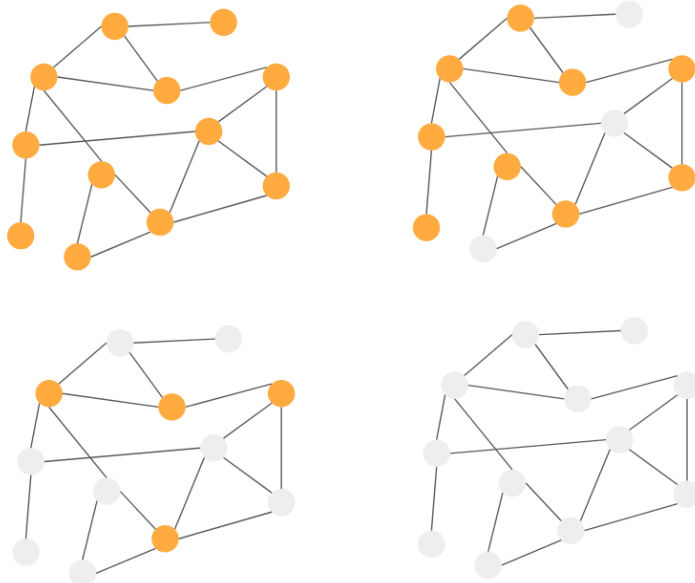


Figure 2: Progressively removing nodes from a network.

We will be interested in percolation on more complicated networks than a grid, and as before, the behaviour of random networks and ones with long tailed degree distributions will be remarkably different. In our network analysis we will usually view the process as taking away nodes from a given network, rather than adding nodes to a growing one, see Figure 2. This will be a more natural point of view for the most important question we will ask about percolation - how robust is the networks to removal of nodes, does it stay connected?

9.2 Percolation transition

In percolation language, rather than removing nodes we talk about nodes being **occupied** and we introduce the **occupation probability** ϕ . When $\phi = 1$ all the nodes are occupied i.e. we have the full network and when $\phi = 0$ none are occupied and the network is empty. The goal is to find the critical value $\phi = \phi_c$ indicating the **percolation transition** from a network that has a single giant component to one that is made up of many small components. For large networks this turns out to be a very sharp transition, so ϕ_c makes sense.

Let u be the probability that a node is **not** connected to the giant com-

ponent via one of its links. If it has degree k then the probability of it not being connected is u^k . Averaging over all degrees

$$g_0(u) = \sum_k p_k u^k \quad (1)$$

is the expected probability of not being in the giant component and so $1 - g_0(u)$ is the probability of being in it (the reason for the subscript 0 will be explained shortly). We multiply this by ϕ to get

$$S = \phi(1 - g_0(u)) \quad (2)$$

the fraction of nodes in the giant cluster. Now we need to calculate u .

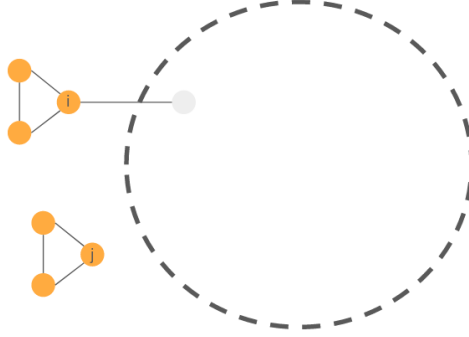


Figure 3: The giant component represented by the dashed boundary. The node i was connected to it along a single link, but the node it connected to was removed. The node j is not connected to it by either of its links.

Recall u is the probability of *not* being connected to the giant cluster *via a particular link*. This can happen in two ways, see Figure 3,

1. The node it connected to has been removed with probability $1 - \phi$
2. The node it connects to is not removed, but is not in the giant cluster. This has probability ϕu^k i.e. the node is present with probability ϕ and none of its k links are connected to the giant cluster with probability u^k .

The total probability is the sum of these conditions

$$1 - \phi + \phi u^k$$

and recalling that the k here is the *neighbouring degree* we average over all k with q_k not p_k so

$$u = \sum_k (1 - \phi + \phi u^k) q_k = 1 - \phi + \phi \sum_k q_k u^k = 1 - \phi + \phi g_1(u) \quad (3)$$

For some particular types of network we can calculate g_0 and g_1 and solve this. In general $u = 1$ is always a solution because $g_1(1) = 1$ in which case

$$S = \phi(1 - g_0(1)) = 0$$

Differentiating both sides of $u = 1 - \phi + \phi g_1(u)$ gives that

$$\frac{dg_1(u)}{du} = g'_1(u) = \frac{1}{\phi}$$

We know the giant component vanishes at $u = 1$ so we define

$$\phi_c = \frac{1}{g'_1(1)} \quad (4)$$

We can write this in terms of things we've seen before. Using $q_k = \frac{(k+1)p_k}{\langle k \rangle}$

$$\begin{aligned} \left. \frac{dg_1}{du} \right|_1 &= \left. \frac{d}{du} \sum_k p_k \frac{(k+1)}{\langle k \rangle} u^k \right|_1 \\ &= \sum_k p_k \frac{(k+1)k}{\langle k \rangle} 1^{k-1} \\ &= \sum_k p_k \frac{(k+1)k}{\langle k \rangle} \end{aligned}$$

or

$$\phi_c = \frac{\langle k \rangle}{\langle k^2 \rangle - \langle k \rangle} \quad (5)$$

This is the **Molloy-Reed criterion**.

Our two favourite examples are a random network and a long tailed one. For the random network $\langle k^2 \rangle = \langle k \rangle(\langle k \rangle + 1)$ and

$$\phi_c = \frac{\langle k \rangle}{\langle k \rangle(\langle k \rangle + 1) - \langle k \rangle} = \frac{1}{\langle k \rangle}$$

This make the network pretty robust. For a random network with average degree 4 we have to remove 3/4 of its nodes to cause it to break apart. A scale free network is even more robust. If $\langle k^2 \rangle \rightarrow \infty$ then $\phi_c \rightarrow 0$, suggesting we can remove basically all of the vertices without breaking it apart.

9.3 Targeted attacks

While scale free networks might be very tolerant to the random removal of nodes, what about non-random removal? Say we deliberately target the ‘hubs’ that make $\langle k^2 \rangle$ diverge and are responsible for keeping the network together?

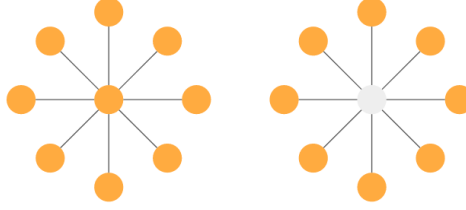


Figure 4: Removal of a hub can have large effects on the connectivity of the network.

To study this we make the occupation probability depend on degree, $\phi \rightarrow \phi_k$. Recall u is the probability to not be connected to the giant cluster via one link. For a given node of degree k the probability to not be connected is u^k and therefore the probability to be connected is $1 - u^k$. That node will have degree k with probability p_k and be present with probability ϕ_k and we sum over all degrees to get the expected probability of being in the giant cluster

$$S = \sum_k p_k \phi_k (1 - u^k) = \sum_k p_k \phi_k - \sum_k \phi_k p_k u^k = f_0(1) - f_0(u) \quad (6)$$

When $\phi_k = \phi$ we get the same expression as before, but with different functions of k we can try different ‘attack strategies’ e.g. preferential removal of high degree nodes.

We can follow the same steps as before to get an equation for u in terms of ϕ_k instead of ϕ . The main thing to be careful of is to use k or $k + 1$ in the right place. Recall the two cases for a node not to be connected to the giant cluster along an edge

1. The node that it connected to is gone. This missing node has degree k with probability q_k and is gone with probability $1 - \phi_{k+1}$
2. The site is occupied but the neighbour is itself not connected which has probability $\phi_{k+1} q_k u^k$

Combining these gives,

$$u = \sum_k q_k(1 - \phi_{k+1} + \phi_{k+1}u^k) = 1 - f_1(1) + f_1(u) \quad (7)$$

As before, we can put in different degree distributions and formulas for ϕ_k but the general solution doesn't have as nice a form as the Molloy-Reed criterion. The most interesting case is a scale-free distribution

$$p_k = ck^{-\gamma}$$

and a strategy focussed on removing high degree nodes

$$\phi_k = \begin{cases} 1 & k < k_T \\ 0 & \text{otherwise} \end{cases} \quad (8)$$

The above is very hard to solve, so we make some simplifications, first treating everything as continuous and second trying to calculate something easier than S , but which will nonetheless tell us a lot about it, f_E the remaining fraction of edges in the network.

To fix the normalization of the power law set

$$c \int_{k_{min}}^{k_{max}} k^{-\gamma} = 1$$

This gives

$$c = \frac{\gamma - 1}{k_{min}^{-\gamma+1} - k_{max}^{-\gamma+1}}$$

We will use the fact that $k_{max} \gg k_{min}$ to ignore negative powers of k_{max} and also assume $\gamma > 2$. Let f_T denote the fraction of nodes above the attack threshold k_T . We can compute this by integrating the degree distribution above k_T ,

$$f_T = \int_{k_T}^{k_{max}} ck^{-\gamma} = c(\gamma - 1)(k_T^{-\gamma+1} - k_{max}^{-\gamma+1}) \quad (9)$$

If we assume also that $k_{max} \gg k_T$ then, ignoring all the k_{max} terms

$$f_T = \left(\frac{k_T}{k_{min}} \right)^{-\gamma+1} \quad (10)$$

or inverting

$$k_T = k_{min} f_T^{\frac{1}{\gamma-1}} \quad (11)$$

is the maximum degree when we have removed a fraction ϕ_T of the highest degree nodes. By removing a fraction f_T of the high degree nodes, we remove a fraction f_E of the edges. We can calculate the fraction of removed links using the excess degree distribution:

$$f_E = \int_{k_T}^{k_{max}} \frac{k p_k}{\langle k \rangle} \simeq \frac{c}{\langle k \rangle (\gamma - 2)} k_T^{-\gamma+2} \simeq \frac{(\gamma - 1)}{\langle k \rangle (\gamma - 2)} \frac{k_T^{-\gamma+2}}{k_{min}^{-\gamma+1}}$$

For a power law

$$\langle k \rangle = c \int_{k_{min}}^{k_{max}} k^{-\gamma+1} \simeq \frac{\gamma - 1}{\gamma - 2} k_{min} \quad (12)$$

substituting gives.

$$f_E = \left(\frac{k_T}{k_{min}} \right)^{-\gamma+2} = f_T^{\frac{2-\gamma}{1-\gamma}} \quad (13)$$

For γ close to 2 we have $f_E \simeq 1$ for any f_T i.e. removing even a small fraction of the hubs can remove a large fraction of the edges, and hence destroy the giant cluster.

9.4 Temporal Networks

A different take on time varying networks is a **temporal network**. This is a set of vertices V and edges E as usual but now the edges are triples

$$(i, j, t)$$

where $i, j \in V$ and t is the time parameter. This time varying edge set gives the **sequence of contacts**. The edges are often given as a 4-tuple

$$(i, j, t, \lambda)$$

where λ is the duration, i.e. how long the edge is ‘on’. We will assume discrete time and represent duration by multiple temporal edges

$$(i, j, t), (i, j, t + 1), (i, j, t + 2), \dots$$

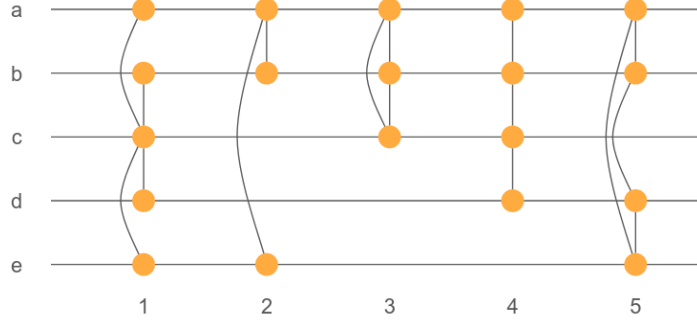


Figure 5: Temporal network represented as a sequence of interactions.

We can represent time varying networks either by animation or on ‘sheet music’ graphs like Figure 5.

A similar idea is to view a temporal network as a sequence of graphs $G_t = \{V_t, E_t\}$, again assuming discrete time steps. Usually $V_t = V$ is constant and only the edges change (opposite to the percolation examples we have just been looking at).

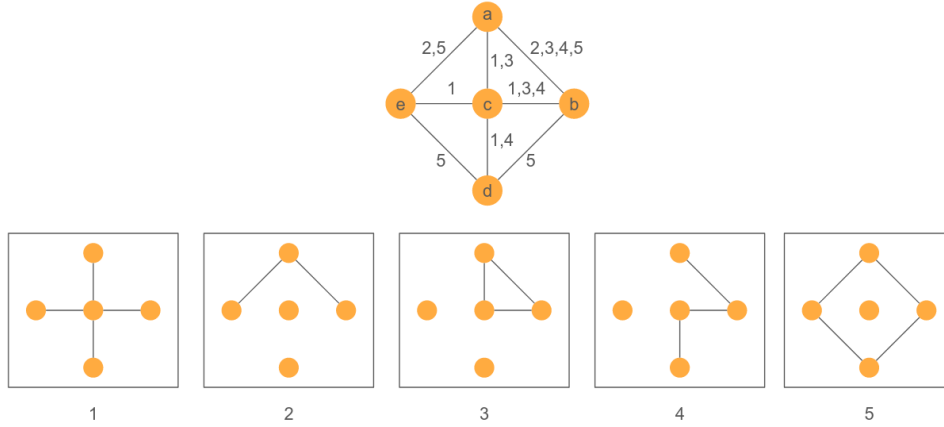


Figure 6: Temporal network represented as a sequence of static networks.

Figure 6 shows a representation of the same temporal network as a *stream* of static networks, as well as a condensed version where the labels on the edges show the timesteps at which the links are present.

A useful idea is the **edge stream**. Here we imagine the sequence of edges in G ‘arriving’ in the order they are created. If two or more edges are

created at the same time, then their order of arrival can be arbitrary. We can also extend the time dependence to nodes as well as links, so that the vertices now ‘appear’ only at particular times i.e. the vertex list is a list of pairs (n, t) where n is the node and t is the time. An edge (i, j, t) implies that the vertices (i, t) and (j, t) exist!

The other models we have been looking at, like the BA model or percolation model, have a network changing over time. This could be thought of as a temporal graph in the ‘sequence of static graphs’ representation. However these are sometimes called **dynamic networks** to distinguish them from temporal ones. The idea with the dynamic graphs we have studied is to look at snapshots of the network and ask about properties of the network in each snapshot, for example, degree distribution. Temporal graph analysis on the other hand looks at properties that span some time interval.

9.5 Expansion into a directed network

Before we move on to looking at metrics and algorithms specific to temporal networks, if we can transform a temporal network into a more familiar type of network, we can use the many algorithms which have been developed for static networks. We also want to do this transformation in a way that preserves the key property of time ordering as much as possible. One possibility for doing this is

1. Make a copy of every node for every time it is present
2. Add a directed edge from the $(t - 1)^{th}$ copy to the t^{th} copy of every node
3. Add all the temporal edges

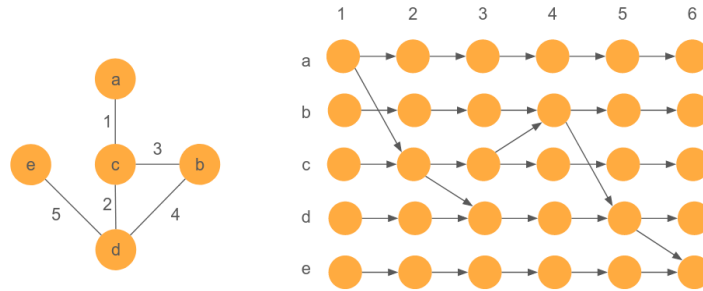


Figure 7: Temporal network represented as a directed network.

An example is shown in Figure 7. Having a static representation allows many classical algorithms for directed networks to be applied, and in this case makes many features obvious, such as the two paths respecting the arrows from a to e , or indeed the fact that any node can be reached starting from a and only a . However one can also see an issue with this representation, the number of nodes grows tremendously!

There is another representation which is like the dual we studied for spatial networks, where nodes are temporal edges connected by shared nodes.

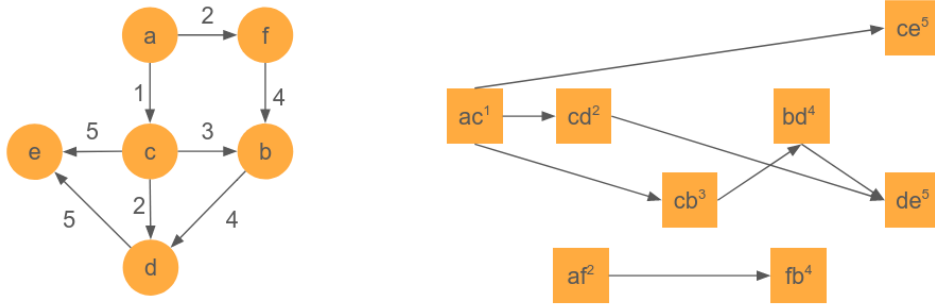


Figure 8: Directed line graph.

A **directed line graph** represents a directed temporal graph using a node for each directed temporal edge and joining them if they share a node. This is quite a bit more efficient than the previous representation!

9.6 Temporal Walks

A **temporal walk** is a sequence of edges where

- consecutive edges share a common node
- the timestamp of edges in the sequence is increasing

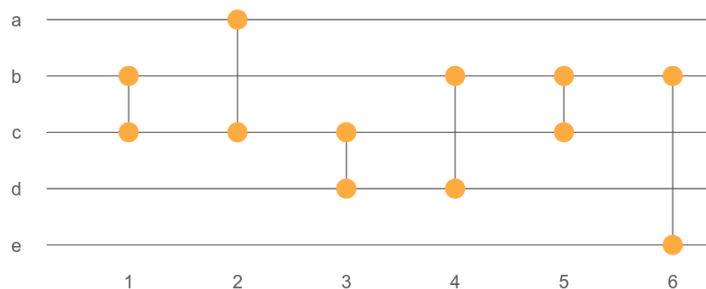


Figure 9: Temporal network represented as a sequence of interactions.

In the rather sparse temporal network in Figure 9 there are several examples such as

$$(b, c, 1), (c, d, 3), (d, b, 4), (b, e, 6)$$

This is a temporal walk. Note that

1. We can *wait* at a node, we don't have to move every time step. One could add self-loops like $(c, c, 2)$ for every node at every time to represent this, but this is a bit cumbersome.
2. We can backtrack on the nodes but not through time!

Diseases, information, water and so on can only flow on temporal walks. We say v is **temporally reachable** from u if there is a temporal walk from u to v . Of course there may only be such a walk within a given interval, so reachability depends on when and how long we are waiting for.

With this idea of temporal walks and reachability we can ask about exploration, can we reach *every* node in G via a temporal walk starting from u ? This is the equivalent to the Hamiltonian path problem on regular graphs, since, by definition, we can only use each temporal edge once. There are lots of other ideas from the analysis of static networks that have generalisations to temporal networks where the restrictions of temporal networks introduce additional complications.

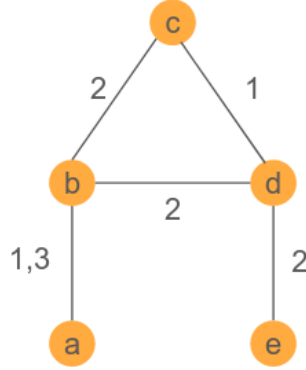


Figure 10: Temporal network with several walks.

Figure 10 shows an example of how a static network and temporal one with the same set of edges might nonetheless be quite different. There is a temporal path from c to d but not from d to c . There is a path from a to c via $(a, b, 1), (b, c, 2)$ there is one from c to a via $(c, b, 2), (b, a, 3)$ however there is no path from a to c and back to a .

9.7 Shortest temporal paths

In analysing static networks the concept of a shortest path is very useful. There are numerous generalisations of this concept to temporal networks. For a walk between u and v we could have

- **earliest arrival path** arrival via an edge (j, v, t) with the smallest possible t ,
- **latest departure path** arrival at v leaving from (u, j, t) with the largest possible t ,
- **fastest arrival path** leaving from (u, a, t_i) arriving from (b, v, t_f) where $t_f - t_i$ is minimized,

and many more.

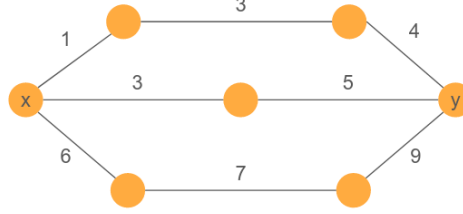


Figure 11: Temporal network with several paths from x to y .

Figure 11 shows all three examples given, the top path is the earliest arrival, the bottom is the latest departure and the middle the the fastest arrival. There are again surprising features of temporal paths which are not the same as standard paths.

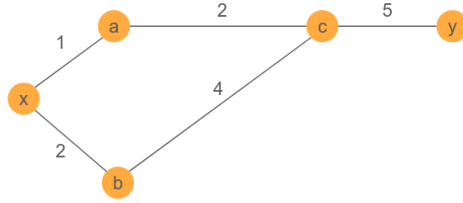


Figure 12: Temporal network with two paths from x to y .

Figure 12 shows a network with two paths from x to y

$$(x, a, 1), (a, c, 2), (c, y, 5)$$

$$(x, b, 2), (b, c, 4), (c, y, 5)$$

The second one is the fastest with duration $5 - 2 = 3$. However note for the sub paths

$$(x, a, 1), (a, c, 2), (c, y, 5)$$

$$(x, b, 2), (b, c, 4), (c, y, 5)$$

The first one is faster (duration 1 versus 2). Thus, unlike shortest paths in standard graphs, a subpath of a fastest path is not necessarily itself a fastest path.

Many of the algorithms which work for static networks do not directly generalise to temporal ones but methods do exist to find them. To compute the earliest arrival path assume we get the network data as an edge stream. For a source node x and a time interval $[t_i, t_f]$ the following process computes the earliest arrival path length.

1. Initialize an array $A[x] = t_i$ and $A[v] = \infty$ for all other nodes $v \neq x$
2. For every edge (u, v, t) in the stream:
 - (a) If $t \geq A[u]$ we already reached u ...
 - (b) ... so we could use the edge to get to v ...
 - (c) Update $A[v] = \min(t, A[v])$

For the network in Figure 12, with nodes in order x, a, b, c, y and the interval $[0, 5]$ the algorithm progresses like

$$\begin{aligned}
 A &= [0, \infty, \infty, \infty, \infty] \\
 A &= [0, 1, \infty, \infty, \infty] \leftarrow (x, a, 1) \\
 A &= [0, 1, \infty, 2, \infty] \leftarrow (a, c, 2) \\
 A &= [0, 1, 2, 2, \infty] \leftarrow (x, b, 2) \\
 A &= [0, 1, 2, 2, \infty] \leftarrow (b, c, 4) \\
 A &= [0, 1, 2, 2, 5] \leftarrow (c, y, 5)
 \end{aligned}$$

At the end A contains the earliest arrival times from x to any node in the network reachable in the given interval. If we keep track of some extra information we can also reconstruct the paths themselves.

This algorithm can be modified to compute the latest arrival path (by streaming the links in backwards!) or more interestingly, the fastest arrival path.

1. Initialize an array $f[x] = 0$ and $f[v] = \infty$ for all other nodes $v \neq x$
2. Let S be the set of distinct start times from x in the interval.
3. For every t in S :
 - (a) Compute the earliest arrival times using the algorithm above and the interval $[t, t_f]$ to get A_t .
 - (b) For every v update $f[v] = \min(f[v], A_t[v] - t)$

This is really quite simple - we find all the possible starting times at which we could leave x , compute the earliest arrival times from each of these start times, then keep the paths which have the smallest sum of time to start +

time to complete. Since we have two possible start times for the network in Figure 12 we have

$$\begin{aligned} A_1 &= [0, 1, 2, 2, 5] \\ A_2 &= [1, \infty, 2, 4, 5] \end{aligned}$$

and hence

$$\begin{aligned} f_1 &= A_1 - 0 \\ f_1 &= [0, 1, 2, 2, 5] \\ f_2 &= A_2 - 1 \\ &= [1 - 1, \infty, 2 - 1, 4 - 1, 5 - 1] \\ &= [0, \infty, 1, 3, 4] \\ &= [0, 1, 1, 2, 4] \end{aligned}$$

However this algorithm is quite inefficient, requiring us to process the edge stream multiple times. A much faster algorithm is available, but it is a bit complicated, you can read the details in the paper of Wu et. al. if you want [4].

9.8 Temporal centrality

We discussed many ways of identifying important nodes for static networks. There are generalisations of these to temporal networks. Temporal networks have a very definite direction, forward in time, and our focus is on temporal coherence of structures over time. Temporal centrality measures should therefore capture either **radial** properties, which measure the influence of a node as the origin or destination of many temporal paths, or **medial** properties, measuring the influence of a node as a ‘mediator’ i.e. its presence on many temporal paths.

A measure which captures radial importance is **closeness centrality**. For static networks we defined this as

$$C_i = \frac{1}{N-1} \sum_{i \neq j} \frac{1}{d_{ij}} \quad (14)$$

where d_{ij} is the length of a shortest path between nodes i and j . This is slightly different than the definition we gave for static networks, there are

often many small variations in the definition of network metrics. Regardless, one obvious generalisation to temporal networks is to replace shortest paths with fastest arrival paths. Now d_{ij} is the minimal duration, i.e. $t_{arrive} - t_{start}$ to send a message between i and j . In this case C_i identifies nodes which spread information fast. We could also use the earliest arrival path duration for d_{ij} . In which case C_i would identify nodes which spread information soonest. These may not be equivalent, so you have to think carefully about what you actually want to measure.

A measure of medial importance is **betweenness centrality**

$$b_i = \sum_{ij} \frac{n_{jk}(i)}{n_{jk}} \quad (15)$$

where n_{jk} is the number of shortest paths between j and k and $n_{jk}(i)$ is the number of shortest paths visiting i . Again the obvious generalization is to replace shortest paths with quickest or earliest arrival paths.

Finally, an interesting generalization of PageRank called **TempoRank** uses the snapshot representation of a temporal network. Let

$$k_i(t) = \sum_j A(t)_{ij} \quad (16)$$

be the degree of i in snapshot t represented by adjacency matrix $A(t)$. Imagine a random walk on the temporal network that stays where it is with probability q and follows a random edge with probability $(1 - q)$. We can define this process with a transition matrix

$$T_{ij}(t) = \begin{cases} \delta_{ij} & k_i(t) = 0 \\ q^{k_i(t)} & k_i(t) > 0, i = j \\ A(t)_{ij}(1 - q^{k_i(t)})/k_i(t) & k_i(t) > 0, i \neq j \end{cases} \quad (17)$$

This encodes the three possibilities

1. The walker does not move because there are no edges at t
 2. $k_i(t)$ edges exist, but the walker does not choose any of them
 3. The walker chooses one of the $k_i(t)$ edges at random and moves there
- . We can iterate this matrix to get a 1, 2, 3, etc. step random walk

$$T^{(p)} = T(t_1)T(t_2) \dots T(t_p)$$

and use eigenvectors of this matrix to define something similar to PageRank, see Chapter 4 of the book [5] for all the details.

9.9 Temporal communities

There are *many* definitions of a community in a static network. Adding time dependence only expands the possibilities. Communities can undergo numerous different processes as the network changes

- Birth
- Death
- Merging
- Splitting
- Growth
- Contraction

and more

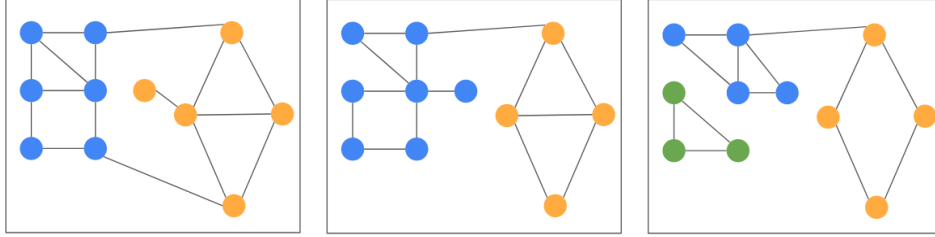


Figure 13: Temporal network communities. A node swaps communities in the second panel growing the blue and contracting the orange. Then the blue community splits in the third panel.

Figure 13 shows some examples. The most obvious approach to studying this is to use the snapshot representation viewing the temporal network as a series of static networks. We can then apply some standard community detection method on each snapshot. This results in several communities for each snapshot $C_i(t)$. We then need to figure out the best matching between snapshots, i.e. which community at step $t + 1$, if any, corresponds to the communities identified in steps $\dots, t - 2, t - 1, t$. To identify similarity, we could use something like the **Jaccard index**

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|} \quad (18)$$

We can identify persistent communities between steps if their Jaccard index is sufficiently big. Generally, to apply this method we need to assume that the time resolution is relatively high and most of the nodes stay in the same community between steps.

A more interesting generalisation is the idea of **episodes**. An episode is an interval $I = [t_1, t_2]$ together with a subgraph $H \subset G[I]$, where $G[I]$ is the network induced by the set of edges which are present during the interval I . A good episode is one which is **dense** i.e. it has lots of connections between the nodes.

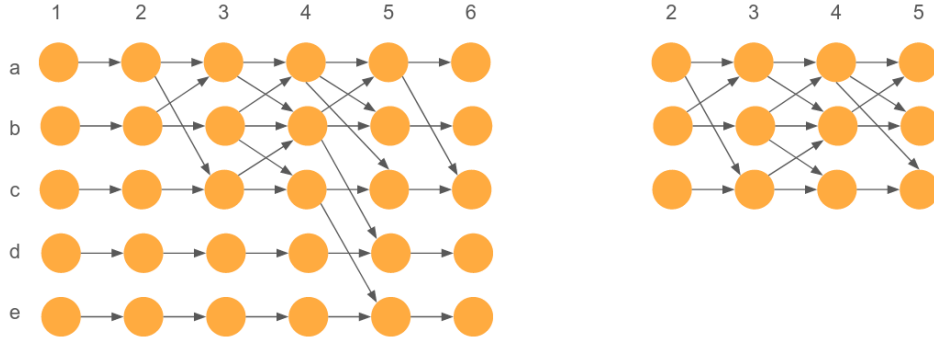


Figure 14: Temporal network and a dense subgraph (episode) consisting of the nodes a, b, c in the interval $[2, 5]$.

Define

$$\rho_H = \frac{|E_H|}{|V_H|} \quad (19)$$

the density of H . The aim is to split the induced temporal network $G[I]$ into episodes so as to maximise the density in each one i.e. find K (given) disjoint episodes that maximise

$$\sum_i^K \rho_{H_i}$$

Algorithms exist to optimise this, but tend to be quite complicated, have a look at the further reading for more information.

References

- [1] Márton Pósfai and Albert-László Barabási. *Network science*. Cambridge University Press, 2016.

- [2] Mark Newman. *Networks*. Oxford university press, 2018.
- [3] Mining temporal networks. <https://miningtemporalnetworks.github.io/>. Accessed: 2025-06-12.
- [4] Huanhuan Wu, James Cheng, Silu Huang, Yiping Ke, Yi Lu, and Yanyan Xu. Path problems in temporal graphs. *Proceedings of the VLDB Endowment*, 7(9):721–732, 2014.
- [5] Naoki Masuda and Renaud Lambiotte. *A guide to temporal networks*. World Scientific, 2016.